# 4.2 Fundamentals of data structures

## 4.2.1 Data structures and abstract data types

### 4.2.1.1 Data structures

| Content | Additional information |
|---|---|
| Be familiar with the concept of data structures. | It may be helpful to set the concept of a data structure in various contexts that students may already be familiar with. It may also be helpful to suggest/demonstrate how data structures could be used in a practical setting. |

### 4.2.1.2 Single- and multi-dimensional arrays (or equivalent)

| Content | Additional information |
|---|---|
| Use arrays (or equivalent) in the design of solutions to simple problems. | A one-dimensional array is a useful way of representing a vector. A two-dimensional array is a useful way of representing a matrix. More generally, an *n*-dimensional array is a set of elements with the same data type that are indexed by a tuple of *n* integers, where a tuple is an ordered list of elements. |

### 4.2.1.3 Fields, records and files

| Content | Additional information |
|---|---|
| Be able to read/write from/to a text file. | |

| Content | Additional information |
| --- | --- |
| Be able to read/write data from/to a binary (non-text) file. | |

### 4.2.1.4 Abstract data types/data structures

| Content | Additional information |
| --- | --- |
| Be familiar with the concept and uses of a:<br>• queue<br>• stack<br>• graph<br>• tree<br>• hash table<br>• dictionary<br>• vector. | Be able to use these abstract data types and their equivalent data structures in simple contexts.<br>Students should also be familiar with methods for representing them when a programming language does not support these structures as built-in types. |
| Be able to distinguish between static and dynamic structures and compare their uses, as well as explaining the advantages and disadvantages of each. | |
| Describe the creation and maintenance of data within:<br>• queues (linear, circular, priority)<br>• stacks<br>• hash tables. | |